总体验收口径（项目完成的定义）

GenUI Project Specification

2026 年 1 月 28 日

系统闭环定义

系统需满足以下闭环流程：

自然语言需求 → 生成 JSON DSL → 约束校验/修正（基于 datasheet/RAG）
→ 确定性渲染 → 可预览/可导出 → 全链路日志与指标可复现

并且满足以下核心要求：

- **安全性拦截：**任何可能损坏硬件的参数必须被拦截或夹紧（clamp/reject），且可解释（来源、规则、修正前后）。
- **论文级记录：**每次运行都能产出可用于论文的结构化记录：违规率、修正率、成功率、延迟、可用性等。

1 基线固化与工程卫生（1 个里程碑：能稳定跑）

目标：你现有 GenUI 不改思路，先让它稳定、可回归、可测，作为对照组（Baseline）。

1.1 依赖/结构完整性检查

- **任务：**补齐缺失模块、统一入口、整理 requirements、加 `scripts/health_check.py`。
- **验收：**
  - `python -m py_compile` 全通过。
  - `scripts/health_check.py` 输出 PASS 并给出 `run_dir`（或 artifacts 目录）。
  - Streamlit 能启动且至少完成一次生成 → 预览 → 导出。

1.2 运行产物规范化（为论文留痕）

- **任务：**定义 `run_dir/` 结构（input、model_output、lint_report、final_html、timing、errors）。
- **验收：**
  - 每次运行都会落盘一个独立目录。
  - 目录包含：输入 prompt、模型原始输出、修复记录、最终产物、耗时统计。

1.3 对照组冻结（Baseline 版本标签）

- **任务：** 把“直接生成 HTML + lint + self-repair”作为 Baseline 分支/开关保留。
- **验收：**
  - UI 有开关：`baseline_html_mode = True/False`。
  - Baseline 模式在后续迭代中不被破坏（回归用例固定）。

论文需要的产出（Phase 0 积累）

- 运行日志（成功/失败/修复次数/耗时）。
- Baseline 的失败案例库（语法错、缺资源、参数幻觉导致危险设置等）。

2 DSL（JSON）与确定性渲染（系统从“文本 →HTML”跃迁为“文本 →DSL→UI”）

目标： 把 LLM 的输出空间从 HTML 缩到可验证的 DSL；渲染完全确定性（Deterministic Rendering）。

2.1 DSL Schema（Pydantic + JSON Schema）

- **任务：** 实现 `modules/schema.py`：Panel / Widget（Slider、Switch、Gauge）/ Layout / Binding / SafetyPolicy / Metadata。
- **验收：**
  - 任意 DSL 必须通过 Pydantic 校验，否则拒绝进入渲染。
  - `schema.json()` 可导出 JSON Schema（用于论文/附录）。
  - 对非法字段、越界类型、缺失字段有明确报错。

2.2 确定性渲染器 Renderer

- **任务：** 实现 `modules/renderer.py`：DSL → HTML（或 Streamlit component）。
- **验收：**
  - 同一 DSL 输入，多次渲染输出字节级一致（可用 hash 验证）。
  - Renderer 不依赖 LLM、不做“智能修复”。
  - 导出的 HTML 可在浏览器直接打开（无外部后端也可运行 UI）。

2.3 Architect Prompt 切换为只产 DSL

- **任务：** 新增 DSL 模式 prompt（few-shot 强制 JSON 输出），保留 Baseline。
- **验收：**
  - DSL 模式下，模型输出必须是 JSON（不允许夹带 HTML）。
  - JSON 解析失败必须触发“要求重写 JSON”的修复回合（最多 N 次）。
  - DSL → 渲染 → 预览链路可跑通。

论文需要的产出

- DSL 示例库（按复杂度分级）。
- “确定性渲染”对比图：Baseline 语法失败率 vs DSL 模式失败率（至少记录 30–100 次运行）。

3 物理约束模型（Constraint Model）与 Verifier（符号校验核心）

目标： 把“安全”变成系统的一等公民：可校验、可修复、可解释。

3.1 约束对象模型（Constraint Model）

- **任务：**实现 `modules/constraints.py`(或并入 verifier)。Constraint：range/max/min/enum/regex/unit/sou
- **验收：**
  - 约束能序列化为 JSON（可作为数据集）。
  - 支持单位与简单换算（至少 V/mV、A/mA、Hz/kHz）。

3.2 Verifier：DSL 参数校验与修复策略

- **任务：** 实现 `modules/verifier.py`。
  - 输入：DSL + constraints
  - 输出：VerificationReport（violations, fixes, residual_risks, citations）
  - 策略：CLAMP / REJECT / REQUIRE_CONFIRM（至少前两者）
- **验收：**
  - 人为构造的危险 DSL（超压/超电流/超 PWM 频率/非法协议字段）必被拦截或夹紧。
  - Report 必含：rule_id、source_ref、before/after、reason。
  - 全链路落盘：`verifier_report.json`。

3.3 “闭环修复”集成（Neuro-Symbolic）

- **任务：**当 verifier 报错时，系统用结构化反馈让 LLM 修改 DSL（不是修改 HTML）。
- **验收：**
  - 触发违规 → LLM 修复 → 再次验证 → 通过 → 渲染成功闭环。
  - 修复次数受控（例如 ≤ 2 次）。
  - 每轮修复保留 diff（用于论文）。

论文需要的产出

- 违规类型分布（电压/电流/协议/频率）。
- 修复成功率、平均修复轮数、修复前后参数分布图。
- 可解释性示例（表格：违规 → 规则 → 修复 → 引用）。

4 Datasheet RAG：从 PDF 自动抽约束（把“人写 constraints.json”变成“机器读 datasheet”）

目标：让约束来源可追溯：从 PDF 文档来，不靠拍脑袋。

4.1 PDF 摄取与证据定位

- **任务：**实现 datasheet ingestion（文本切分、索引、检索）；返回证据：页码/段落/表格标题（能定位）。
- **验收：**
  - 给定一个 datasheet PDF，能检索到“Absolute Maximum Ratings / Recommended Operating Conditions”等段落。
  - 输出 evidence payload：page、span、snippet（落盘）。

4.2 约束抽取器（Extractor）

- **任务：**从检索到的证据中抽出结构化约束（可先规则 +LLM）。
- **验收：**
  - 至少稳定抽出：供电电压范围、I/O 电压、最大电流、通信速率范围（取决于你选的设备）。
  - 抽取失败要有降级策略：允许人工补一个最小 `constraints.json`，但必须标注为 manual。

4.3 约束一致性与置信策略

- **任务：** 同一参数多个来源冲突时如何处理（取更保守/标记冲突）。
- **验收：**
  - 冲突会生成 `constraint_conflicts.json` 并默认选择保守值。
  - 运行不会默默忽略冲突。

论文需要的产出

- 抽取准确率（人工标注少量样本即可）。
- RAG vs manual constraints 的对比（覆盖率、冲突率、错误率）。
- 证据可追溯截图/引用（页码级）。

5 工业协议与“可连接”的 HMI（从 UI Demo 到 IIoT HMI）

目标： 不仅安全，还能“真的连设备/模拟设备”，同时不牺牲可验证性。

5.1 Binding 层：MQTT / Modbus（先选一个主线）

- **任务：** 实现 `modules/bindings/`：协议配置、topic/register 映射、连接参数（仍由 DSL 描述）。
- **验收：**
  - DSL 能描述 binding（host、port、topic/register、QoS 等）。
  - 在无真实设备下，提供 simulator/mock（保证 CI 可跑）。

5.2 安全策略下推到运行时（Runtime Guard）

- **任务：** 运行时发送命令前再过一层 guard（防止 UI 注入或外部修改绕过）。
- **验收：**
  - 任意来自 UI 的 command 都经过 guard。
  - guard 违规必阻断并记录事件（node_event/log）。

5.3 事件与状态模型（为 HMI 可观测性）

- **任务：** 统一状态流：telemetry/command/ack/error（可用 JSON event）。
- **验收：**
  - 事件流可回放（replay），同一事件序列渲染结果一致。
  - 能导出一份 session log（论文里可作为案例）。

论文需要的产出

- 端到端延迟统计（UI → 协议 → ack）。
- 运行时违规拦截次数（与生成时 verifier 的互补）。
- Demo case study：一个设备（真实或模拟）从 datasheet → 生成 HMI → 安全运行。

6 系统级评估与可投稿证据链（不以写作为主，但把图表都“长出来”）

目标：你不主写论文，但必须让论文“只差组装”。

6.1 Benchmark 任务集（Prompt Suite）

- **任务：**建立 30–100 条需求语句，覆盖不同设备/不同控件/不同协议/不同危险边界。
- **验收：**
  - 每条 prompt 有期望的 DSL 特征与安全约束点（gold check）。
  - 一键跑完整套并产出总报告。

6.2 指标体系与自动出图

- **任务：**metrics + plotting 脚本（不依赖手工）：合法 DSL 率、渲染成功率、违规率、修复率、平均修复轮数、证据可追溯率、端到端耗时。
- **验收：**
  - `python scripts/run_benchmark.py` 输出 `report.json + figures/`（png/pdf）。
  - 关键曲线自动生成：成功率对比、违规类型饼图/柱状、修复轮数分布、耗时分解。

6.3 Ablation（消融）开关

- **任务：**至少 4 个开关：`baseline_html_mode`、`dsl_mode_no_verifier`、`verifier_no_rag`（仅手写 constraints）、`rag_no_extractor`（只检索不抽约束）。
- **验收：**
  - 同一 benchmark 在不同开关下可重复运行并输出可比报告。
  - 报告字段固定（便于论文表格）。

论文需要的产出（到此基本齐活）

- 1 张系统架构图（你可以用 DSL/Verifier/Renderer 的模块图）。
- 2–4 张核心实验图（成功率/违规率/修复轮数/耗时）。
- 1–2 个 case study（含 datasheet 引用与修复解释）。

路径建议与门槛清单

你当前到“完全实现”的最短路径建议（按优先级）

1. **Phase0** 做到稳定可回归（否则后面全是漂的）。
2. **Phase1** 的 DSL+Renderer（这是你创新的门槛）。
3. **Phase2** 的 Verifier 闭环（这是“安全”落地的核心）。
4. **Phase3** 的 datasheet → constraints 自动化（这是你“物理感知”的证据）。
5. **Phase4/5** 作为增强与投稿证据链补齐。

最关键的“验收门槛”清单（你每阶段都要过）

- **可复现：** 同一输入 → 同一 DSL → 同一 HTML（hash 一致）。
- **可解释：** 每次修复都有 before/after + rule + source_ref。
- **可回归：** 一键 benchmark + 固定报告字段。
- **可安全：** 生成时 verifier + 运行时 guard 双层防线。